

AIAA 2205: Introduction to AI Natural Language Processing

Li LIU

`avrillliu@hkust-gz.edu.cn`

The Hong Kong University of Science and Technology (Guangzhou)

Content of this course

- Language Model
- Pretrained Word Embedding

Content of this course

- Language Model
- Pretrain Word Embedding

□ Language Model (LM)

□ Describes the probability of a piece of natural language or the probability of the next word given $P(w_1, \dots, w_l)$ or $P(w_{l+1} | w_1, \dots, w_l)$

□ The above two definitions are equivalent (chain rule)

□ $P(w_1, \dots, w_l) =$

$$P(w_1)P(w_2|w_1)P(w_3|w_1w_2) \dots P(w_l|w_1w_2 \dots w_{l-1}) = \prod_{i=1}^l P(w_i | w_{1:i-1})$$

- ❑ Widely used in a variety of NLP tasks

- ❑ Machine translation (word sorting)

$$P(\text{the cat is small}) > P(\text{small the is cat})$$

- ❑ Speech recognition (word selection)

$$P(\text{there are } \textcolor{red}{\text{four}} \text{ cats}) > P(\text{there are } \textcolor{green}{\text{for}} \text{ cats})$$

- The N-gram language model assumes that the probability of the next word is only related to the previous $n-1$ words.

$$P(w_t | w_1 w_2 \dots w_{t-1}) \approx P(w_t | w_{t-(n-1):t-1})$$

- ❑ This assumption is called the Markov Assumption
 - ❑ When $n=1$, the occurrence of the next word is **independent** of its history, and the corresponding unigram is usually denoted as a unigram.
 - ❑ When $n=2$, **the next word only depends on the previous word**, and the corresponding bigram model is also called *first-order* Markov Chain.
 - ❑ The trigram hypothesis ($n=3$) is also called the *second-order* Markov hypothesis.

Increasing power of higher-order n-grams

1 gram	–To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have –Hill he late speaks; or! a more to leg less first you enter
2 gram	–Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow. –What means, sir. I confess she? then all sorts, he is trim, captain.
3 gram	–Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done. –This shall forbid it should be branded, if renown made it empty.
4 gram	–King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in; –It cannot be but so.

Figure 3.4 Eight sentences randomly generated from four n-grams computed from Shakespeare's works. All characters were mapped to lower-case and punctuation marks were treated as words. Output is hand-corrected for capitalization to improve readability.

UR Smoothing of N-gram Language Models

- When n is too large, or when words in the test sentence are not in the vocabulary, the problem of **zero probability** may arise.
- To keep a language model from assigning zero probability to these unseen events, we'll have to shave off a bit of probability mass from some more frequent events and give it to the events we've never seen.

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Figure 3.1 Bigram counts for eight of the words (out of $V = 1446$) in the Berkeley Restaurant Project corpus of 9332 sentences. Zero counts are in gray.

Smoothing of N-gram Language Models

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Figure 3.2 Bigram probabilities for eight words in the Berkeley Restaurant Project corpus of 9332 sentences. Zero probabilities are in gray.

Smoothing of N-gram Language Models

□ Add-One Smoothing (Laplace Smoothing)

$$P(w_i|w_{i-1}) = \frac{C(w_{i-1}w_i) + 1}{\sum_w (C(w_{i-1}w) + 1)} = \frac{C(w_{i-1}w_i) + 1}{C(w_{i-1}) + |\mathbb{V}|}$$

□ Add- δ ($0 < \delta < 1$) smoothing (When the training data is small, Add-One smoothing may give overly optimistic estimates), for the selection of δ , evaluate the model's perplexity by comparing different values, and choose the optimal value.

$$P(w_i|w_{i-1}) = \frac{C(w_{i-1}w_i) + \delta}{\sum_w (C(w_{i-1}w) + \delta)} = \frac{C(w_{i-1}w_i) + \delta}{C(w_{i-1}) + \delta|\mathbb{V}|}$$

Smoothing of N-gram Language Models

	i	want	to	eat	chinese	food	lunch	spend
i	6	828	1	10	1	1	1	3
want	3	1	609	2	7	7	6	2
to	3	1	5	687	3	1	7	212
eat	1	1	3	1	17	3	43	1
chinese	2	1	1	1	1	83	2	1
food	16	1	16	1	2	5	1	1
lunch	3	1	1	1	1	2	1	1
spend	2	1	2	1	1	1	1	1

Figure 3.6 Add-one smoothed bigram counts for eight of the words (out of $V = 1446$) in the Berkeley Restaurant Project corpus of 9332 sentences. Previously-zero counts are in gray.

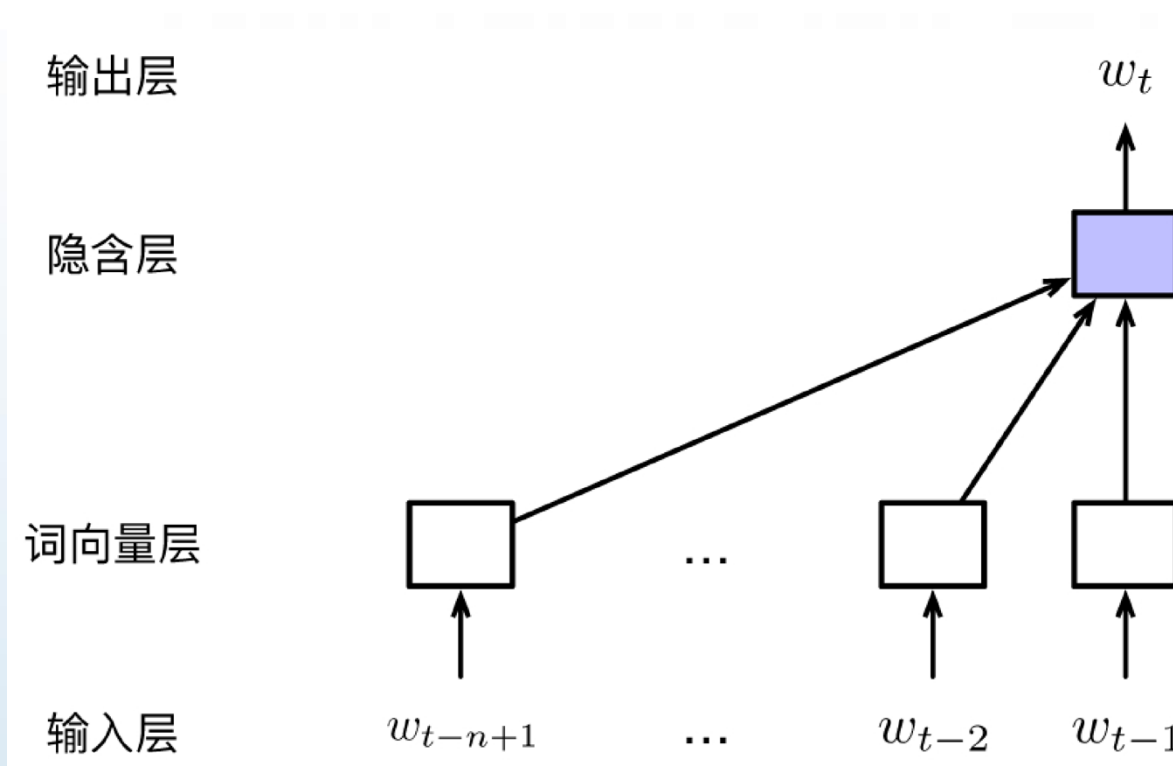
Smoothing of N-gram Language Models

	i	want	to	eat	chinese	food	lunch	spend
i	0.0015	0.21	0.00025	0.0025	0.00025	0.00025	0.00025	0.00075
want	0.0013	0.00042	0.26	0.00084	0.0029	0.0029	0.0025	0.00084
to	0.00078	0.00026	0.0013	0.18	0.00078	0.00026	0.0018	0.055
eat	0.00046	0.00046	0.0014	0.00046	0.0078	0.0014	0.02	0.00046
chinese	0.0012	0.00062	0.00062	0.00062	0.00062	0.052	0.0012	0.00062
food	0.0063	0.00039	0.0063	0.00039	0.00079	0.002	0.00039	0.00039
lunch	0.0017	0.00056	0.00056	0.00056	0.00056	0.0011	0.00056	0.00056
spend	0.0012	0.00058	0.0012	0.00058	0.00058	0.00058	0.00058	0.00058

Figure 3.7 Add-one smoothed bigram probabilities for eight of the words (out of $V = 1446$) in the BeRP corpus of 9332 sentences. Previously-zero probabilities are in gray.

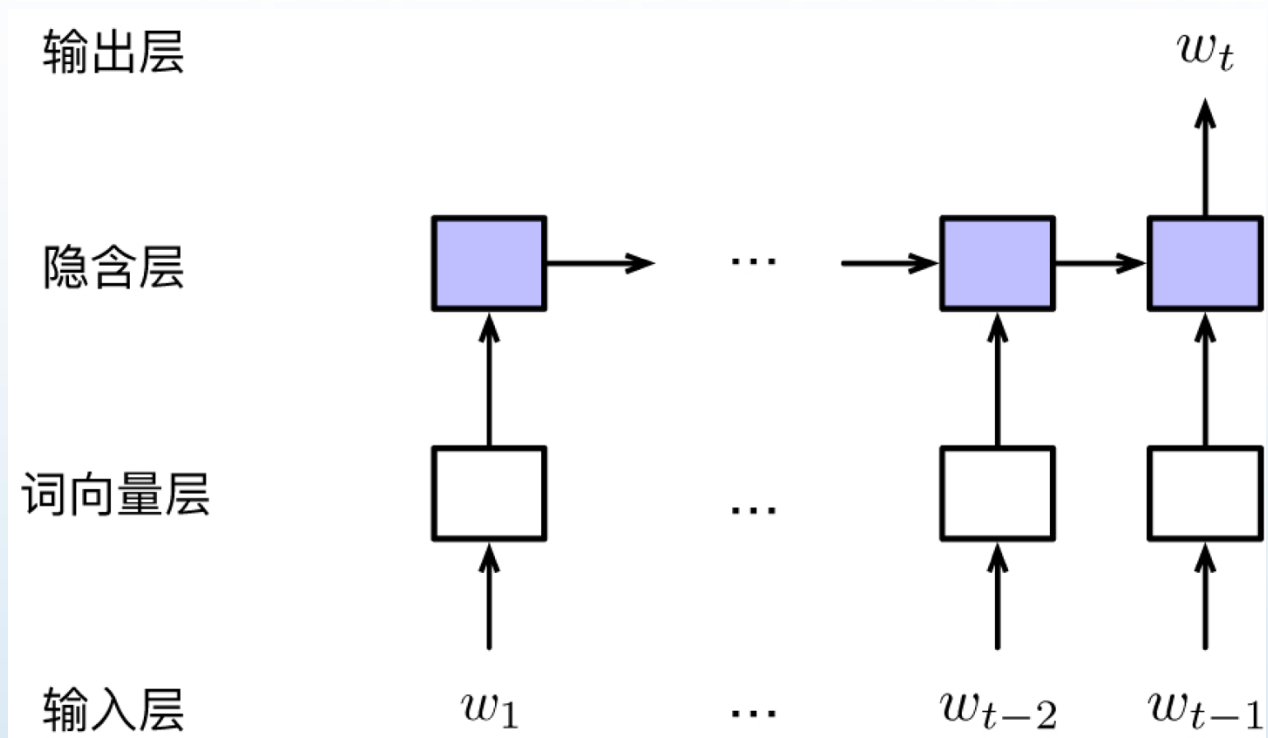
- Estimating conditional probabilities using feedforward neural networks

$$P(w_t | w_{1:t-1}) \approx P(w_t | w_{t-n+1:t-1})$$



- Estimating conditional probabilities using recurrent neural networks

$$P(w_t | w_{1:t-1})$$



1. Input Representation

Each input sentence is first tokenized into subword units (using Byte Pair Encoding or WordPiece). The tokens are then converted into embeddings:

- **Token embeddings:** Learnable vectors representing individual tokens.
- **Positional encodings:** Since the Transformer has no recurrence or convolution, it needs a way to encode token position in the sequence. These can be:
 - *Fixed sinusoidal encodings* (as in the original paper), or
 - *Learnable positional embeddings* (common in modern implementations like GPT).

Final input:

$$\text{Input} = \text{TokenEmbedding} + \text{PositionEmbedding}$$

2. Transformer Encoder or Decoder Blocks

Transformer models for language modeling typically use either:

- **Decoder-only stacks** (e.g., GPT-style, autoregressive), or
- **Encoder-decoder** (e.g., T5, BART), or
- **Encoder-only** (e.g., BERT, for masked LM tasks)

Each block contains:

- **Multi-Head Self-Attention:** Allows each token to attend to other tokens. For autoregressive models, masking is used to prevent seeing future tokens.
- **Feed-Forward Neural Network:** Two-layer fully connected network applied to each position independently.
- **Layer Normalization and Residual Connections:** Aid training stability and gradient flow.

3. Stacking Layers

Multiple identical Transformer layers (e.g., 12, 24, 96) are stacked to increase model capacity.

4. Output Layer

For **language modeling**, the model's goal is to predict the next token given previous tokens:

- A **linear projection** maps the final hidden state to vocabulary size.
- A **softmax** layer converts logits into a probability distribution over tokens.

$$P(\text{next token}) = \text{Softmax}(W \cdot h + b)$$

For **masked language modeling** (e.g., BERT), the model predicts only the masked tokens in the input.

5. Training Objective

Depending on the type:

- **Autoregressive LM (GPT):** Maximize the likelihood of the next token:

$$\max \sum_t \log P(x_t \mid x_{<t})$$

- **Masked LM (BERT):** Predict randomly masked tokens in the input.
- **Seq2Seq (T5/BART):** Learn to generate outputs from inputs (e.g., translate or summarize).

- ❑ External evaluation based on specific applications (closest to actual application requirements, but costly)
- ❑ Internal evaluation based on **perplexity** (PPL)
 - ❑ It indicates the difficulty of predicting the next word given a piece of text.
 - ❑ A lower perplexity indicates that the model is better at predicting the next word, meaning better model performance.
- ❑ Probability of test set

$$P(\mathbb{D}^{\text{test}}) = P(w_1 w_2 \dots w_N) = \prod_{i=1}^N P(w_i | w_{1:i-1})$$

- Perplexity: the geometric mean of the probabilities assigned by the model to each word in the test set.

$$\text{PPL}(\mathbb{D}^{\text{test}}) = \left(\prod_{i=1}^N P(w_i | w_{1:i-1}) \right)^{-\frac{1}{N}}$$

- To avoid underflow (下溢), convert it to exponential logarithmic form.

$$\text{PPL}(\mathbb{D}^{\text{test}}) = 2^{-\frac{1}{N} \sum_{i=1}^N \log_2 P(w_i | w_{1:i-1})}$$

Underflow: result is very close to zero, making the size of the result smaller than the smallest number that can be represented by a floating point number.

Bert language for Q&A

Content of this week

- Language Model
- **Pretrained Word Embedding**

How to represent words?

N-gram language models

It is 76 F and ____.

$$P(w \mid \text{it is 76 F and})$$

[0.0001, 0.1, 0, 0, 0.002, ..., 0.3, ..., 0]
 red sunny

Text classification

I like this movie. 👍

$w^{(1)}$ [0, 1, 0, 0, 0, ..., 1, ..., 1]

I don't like this movie. 👎

$w^{(2)}$ [0, 1, 0, 1, 0, ..., 1, ..., 1]
 don't

Representing words as discrete symbols

In traditional NLP, we regard words as discrete symbols:

hotel, conference, motel — a localist representation

one 1, the rest 0's



Words can be represented by **one-hot** vectors:

hotel = [0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0] Dimension: 16 word

motel = [0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0]

Vector dimension = number of words in vocabulary (e.g., 500,000)

❑ Disadvantage

- ❑ Use high-dimensional, discrete, sparse vectors to represent words.
- ❑ Ignore the context relationship between words.
- ❑ Unable to deal with the phenomenon of “*multiple words with one meaning*”.

□ Distributional semantic hypothesis

□ The meaning of a word can be represented by the distribution of its context words

□ *You shall know a word by the company it keeps* -- Firth J.R. 1957

he curtains open and the moon shining in on the barely
ars and the cold , close moon " . And neither of the w
rough the night with the moon shining so brightly , it
made in the light of the moon . It all boils down , wr
surely under a crescent moon , thrilled by ice-white
sun , the seasons of the moon ? Home , alone , Jay pla
m is dazzling snow , the moon has risen full and cold
un and the temple of the moon , driving out of the hug
in the dark and now the moon rises , full and amber a
bird on the shape of the moon over the trees in front
But I could n't see the moon or the stars , only the
rning , with a sliver of moon hanging among the stars
they love the sun , the moon and the stars . None of
the light of an enormous moon . The splash of flowing w
man 's first step on the moon ; various exhibits , aer
the inevitable piece of moon rock . Housing The Airsh
oud obscured part of the moon . The Allied guns behind

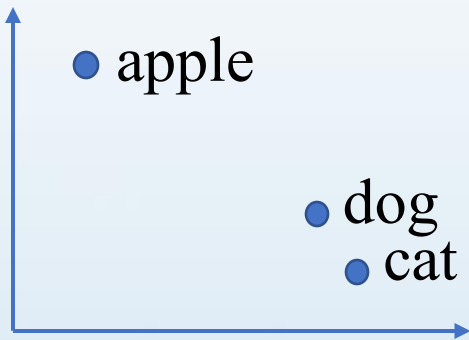
Word Distributional Representation

□ Distributional Word Embeddings

Word	cat	dog	apple	run
food	12	10	1	0
pet	15	14	0	0
fruit	0	0	25	0

Each value represents the number of times the corresponding two words appear together in an article.

□ Semantic similarity is obtained by calculating vector similarity.

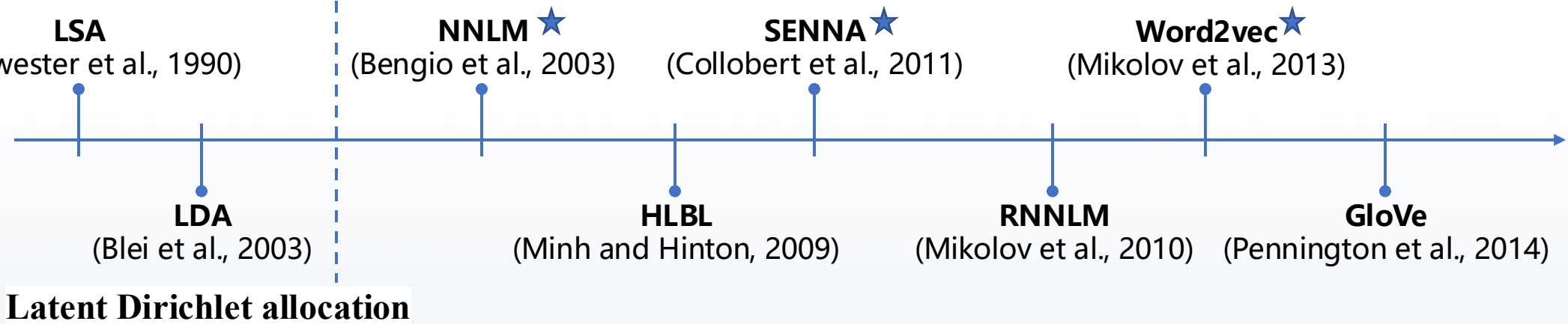


□ The problem of high dimensionality, sparsity, and discreteness still exists.

- ❑ Disadvantages of **distributional representations**
 - ❑ Slow training speed, difficulty in adding new corpora
 - ❑ Challenges in extending to phrase or sentence representations
- ❑ **Distributed representations** directly utilize low-dimensional, dense, and continuous vectors to represent words.
 - ❑ Word Embedding is learned directly through **self-supervised** DNN methods

□ Development history:

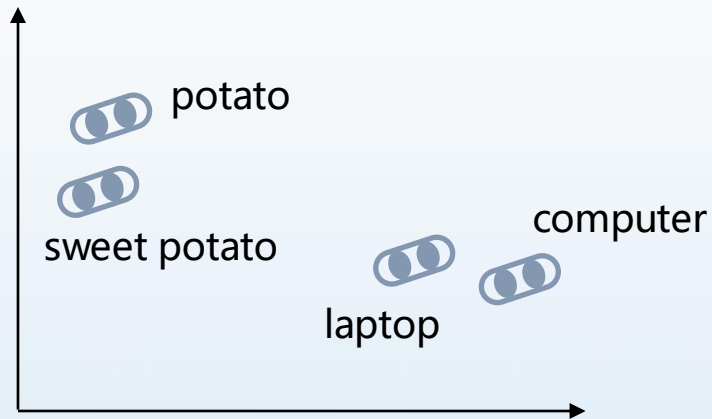
Latent semantic analysis



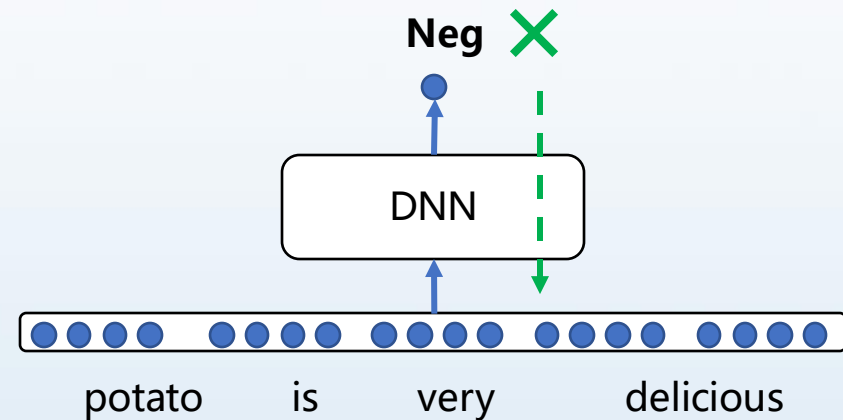
- Neural Network Language Model (NNLM) is a natural language processing method based on neural networks.
- SENNA: Semantic/syntactic Extraction using a Neural Network Architecture.
- Word2Vec by Google is a word embedding model based on neural networks. It has two training methods: CBOW (Continuous Bag-of-Words) and Skip-gram.

Deep learning based on embedding representations

❑ **Word Embedding:** Directly using a low-dimensional, continuous, dense vector to represent words (Bengio et al., 2003)

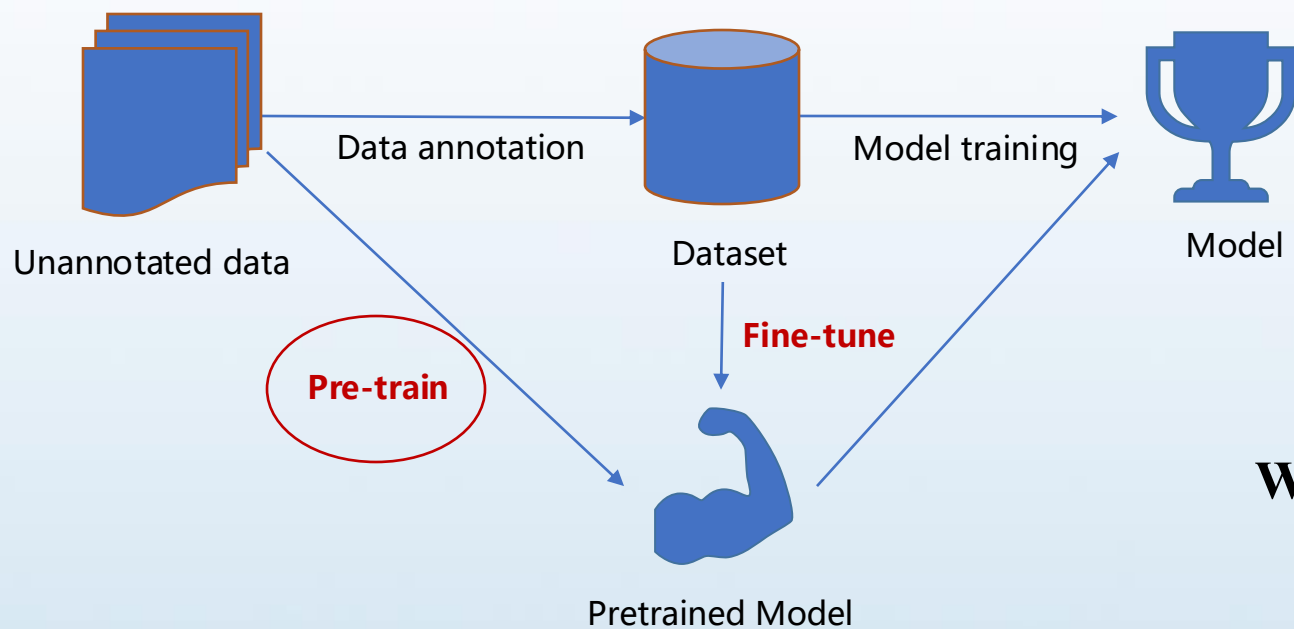
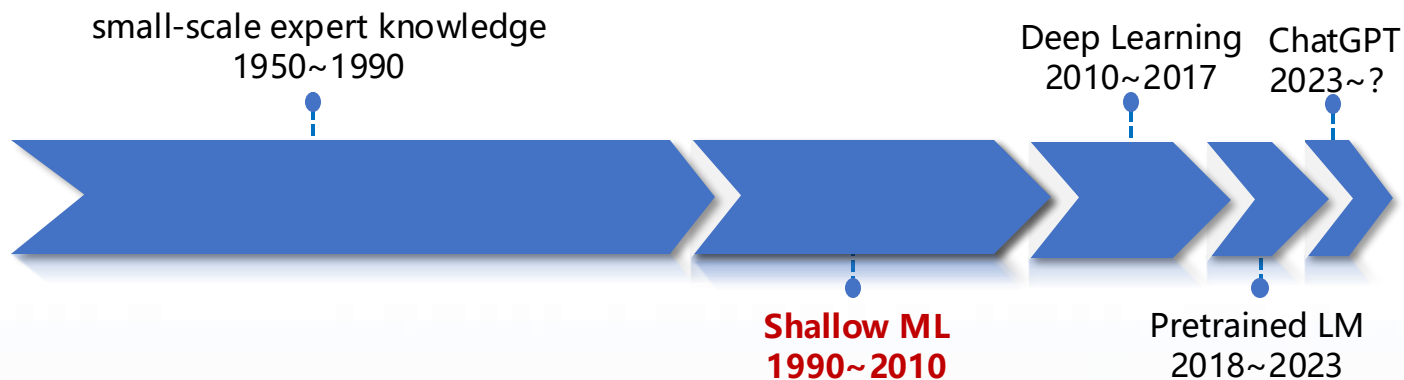


❑ **Assignment methods for word embedding representations:** automatically learning through optimization of performance on downstream tasks





Pretrained Language Model



What is Fine-tune?

UR How to utilize unlabeled data for pre-training?

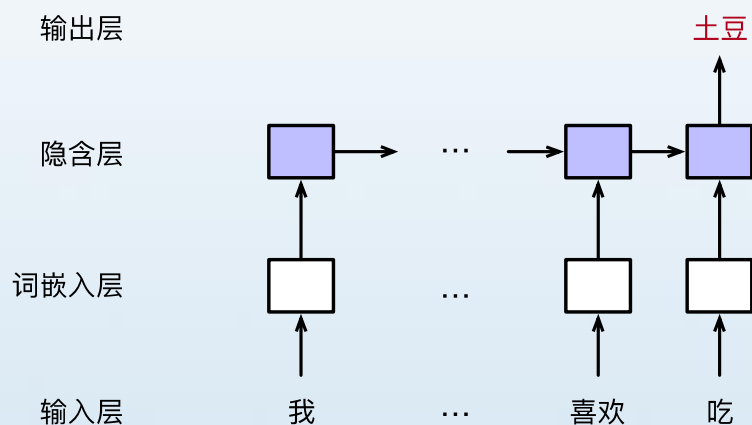
□ Utilizing the inherent sequentiality of language

□ *I like to eat potatoes stew* **XX**

□ Two task types

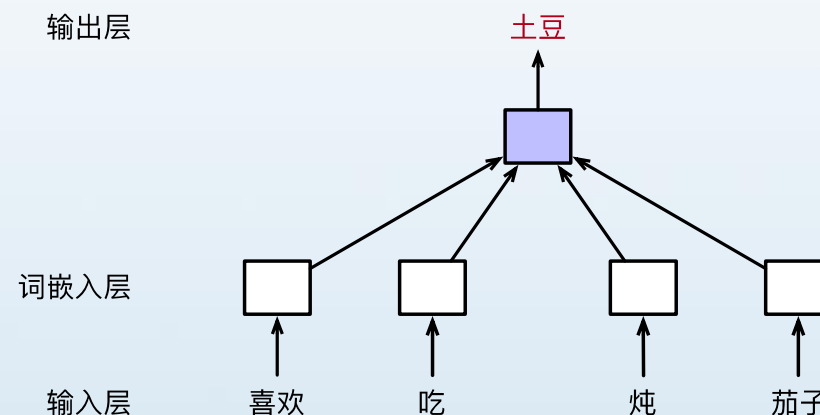
□ Language Model

□ Predicting the next word based on historical word sequences



□ Cloze test (完形填空)

□ Predicting the middle word by surrounding words



- ❑ Neural Network Language Models (Bengio et al., JMLR 2003)
 - ❑ Predict the current word based on the previous $n-1$ words (history), known as the Markov assumption.
 - ❑ The model structure is a feedforward neural network.
 - ❑ Obtain word vector representations through a look-up table.
 - ❑ Optimize word vector representations through gradient descent.

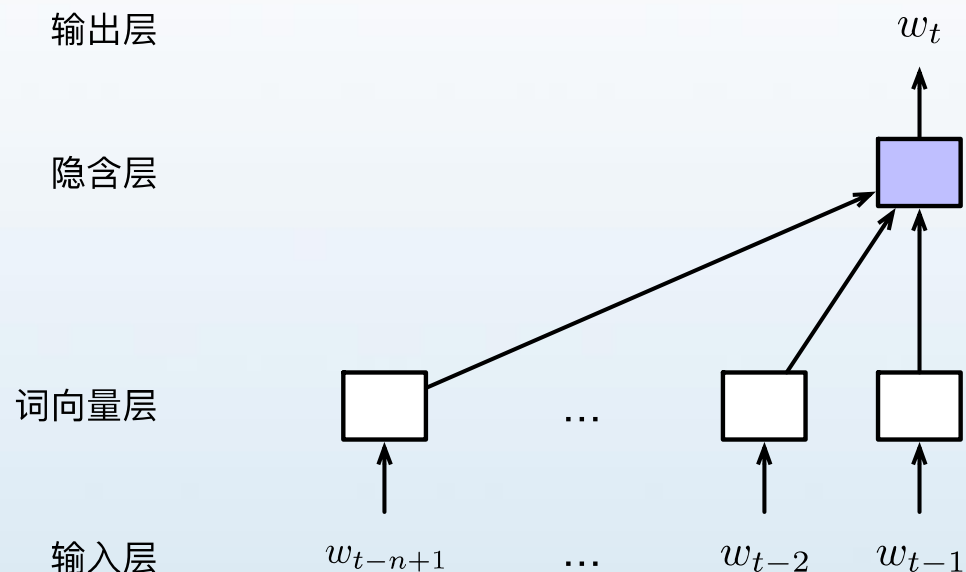


❑ Shortcoming

❑ The length of the history is fixed.

❑ “He likes to eat **apples**”

❑ “He caught a cold, so he went to the **hospital** after get off work”



□ <https://code.google.com/archive/p/word2vec/>

□ Mikolov et al., ICLR 2013

□ Skip-Gram

□ Independently predicting surrounding words (context) based on the middle word.

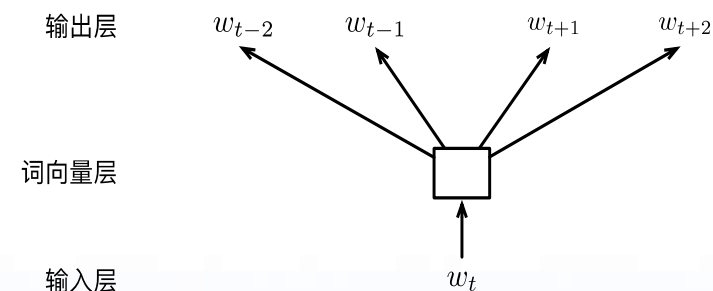
□ Fast training speed

□ CBOW (Continuous Bag-of-Word)

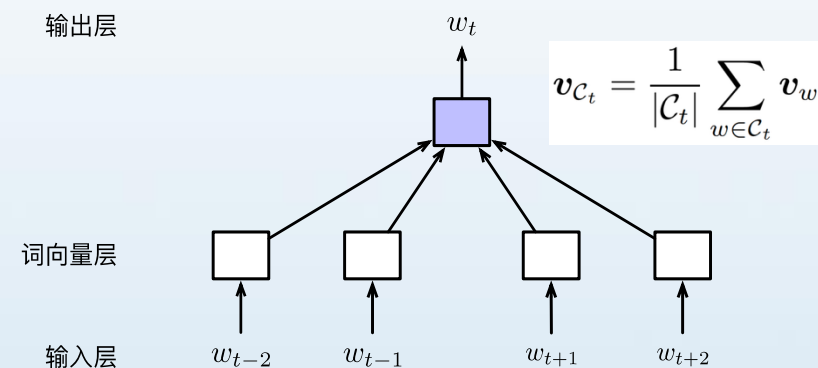
□ Predicting the middle word based on surrounding words (context).

□ How to calculate context representation:

take the average of word vectors.



Skip-gram model



CBOW

Other methods for obtaining Word Embedding

Category	Recommended Methods	Best Use Case
Beginner-friendly	Word2Vec, GloVe	Simple and fast word representations
Most powerful	BERT, RoBERTa	NLP tasks with rich context
Fast/lightweight	DistilBERT, TinyBERT	Deployment and speed-sensitive tasks
Cross-lingual	mBERT, XLM-R	Multilingual NLP
OOV handling	FastText	Better subword support

Let's try Bert for Q&A!

Let's see an example!
FFN-LM



香港科技大學(廣州)
THE HONG KONG
UNIVERSITY OF SCIENCE AND
TECHNOLOGY (GUANGZHOU)



香港科技大學
THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

Thank you!